



3.2. Sudoku Solver Problem

"Logic Unleashed, Puzzles Solved."

Main Section:

3. Backtracking

Contents:

[1. Problem Definition](#)

[2. The Workflow](#)

[Summary](#)

[References](#)

Sudoku is one of the most recognizable puzzle games in the world. At first glance, it looks simple: fill the empty cells with numbers from 1 to 9 while making sure every row, column, and sub-grid remains valid. But once the board becomes difficult enough, solving it is no longer just about intuition. It becomes a search problem.

Humans often solve Sudoku using observation, memory, and pattern recognition. We look for missing numbers, eliminate impossible choices, and slowly narrow down the correct answer. But computers do not *"think"* in that same way. A computer needs a strategy—something systematic that tells it how to explore possibilities and recover when a decision turns out to be wrong.

This is where **Backtracking** becomes important:

- Instead of magically knowing the correct number for every empty cell, the algorithm works by making careful guesses.
- It tries a number, checks whether the placement is valid, and then continues forward.

- If a contradiction appears later, the algorithm does not restart from the beginning. It simply goes backward, removes the incorrect choice, and tries another possibility.

So in a way, a Sudoku Solver is not really about solving the puzzle instantly. It is about exploring possibilities intelligently while respecting the rules of the board. And among all techniques used for this purpose, Backtracking remains one of the most elegant and intuitive approaches.

1. Problem Definition

Sudoku is a logic-based puzzle played on a 9×9 grid consisting of 81 cells divided into nine smaller 3×3 sub-grids. Some cells are already filled with numbers, while the remaining cells are empty.

The objective is to fill every empty cell using numbers from 1 to 9 while satisfying three important conditions:

- **Each number can only appear once in every row**
- **Each number can only appear once in every column**
- **Each number can only appear once in every 3×3 sub-grid**

In other words, every placement must remain consistent with the entire board.

Consider the following Sudoku board:

	1	2	3	4	5	6	7	8	9
A			4		5				
B	9			7	3	4	6		
C			3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

When humans solve Sudoku, they often rely on reasoning and elimination. But for a computer, we need a more systematic process. This is where Backtracking becomes extremely useful.

The core idea of Backtracking is surprisingly simple: ***Try a possible number, move forward if it works, and go backward if it fails.***

The algorithm begins by searching for an empty cell. If the cell is already filled, it simply moves to the next position. But when an empty cell is found, the solver attempts to place numbers from 1 to 9 into that cell.

Of course, not every number is valid. Before placing a number, the algorithm must check whether that number already exists in the current row, the current column, or the current 3×3 sub-grid.

If the number violates any of these conditions, it is rejected immediately. But if the placement is valid, the solver temporarily accepts the number and continues solving the rest of the puzzle.

Later, if the algorithm reaches a situation where no number can be placed into a future cell, it realizes that one of the earlier choices was incorrect. Instead of

starting over completely, it backtracks: the previously placed number is removed, and another possibility is tested.

This process repeats continuously: try, validate, continue, fail, backtrack, and retry, until the entire board is solved.

In Python, a Sudoku board is commonly represented using a 9×9 matrix.

```
sudoku_matrix = [
    [0, 0, 4, 0, 5, 0, 0, 0, 0],
    [9, 0, 0, 7, 3, 4, 6, 0, 0],
    [0, 0, 3, 0, 2, 1, 0, 4, 9],
    [0, 3, 5, 0, 9, 0, 4, 8, 0],
    [0, 9, 0, 0, 0, 0, 0, 3, 0],
    [0, 7, 6, 0, 1, 0, 9, 2, 0],
    [3, 1, 0, 9, 7, 0, 2, 0, 0],
    [0, 0, 9, 1, 8, 2, 0, 0, 3],
    [0, 0, 0, 0, 6, 0, 1, 0, 0]
]
```

Filled cells contain numbers from 1 to 9, while empty cells are represented using 0. The solver then recursively explores every valid possibility until a complete and correct solution is found.

Remember that in programming, indexing starts from 0, so the top-left cell is represented as (0,0).

2. The Workflow

Now let's actually observe how Backtracking solves a Sudoku puzzle step by step, using the above Sudoku board.

The overall procedure of Backtracking is actually very repetitive:

1. Find an empty cell
2. Determine all valid numbers for that cell
3. Try each possible number
4. Continue solving recursively
5. If a contradiction appears, backtrack and try another number

At first glance, this may sound simple. But once recursion begins, the algorithm starts exploring a massive decision tree behind the scenes.

Let's observe the process carefully.

Step 1 — Investigating Cell A1

The first empty cell is A1. Now the solver needs to determine which numbers are feasible for that location.

	1	2	3	4	5	6	7	8	9
A			4		5				
B	9			7	3	4	6		
C			3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

We start with cell A1, which is empty

- First, we inspect **row 0**: $[0, 0, 4, 0, 5, 0, 0, 0, 0]$. Two numbers already exist there: **4, 5**.
- Next, we inspect **column 0**: $[0, 9, 0, 0, 0, 0, 3, 0, 0]$. Two additional numbers exist: **9, 3**.
- Finally, we inspect the top-left **3×3 sub-grid**: $[0, 0, 4][9, 0, 0][0, 0, 3]$. The existing numbers are: **4, 9, 3**.

So now the solver combines all constraints. The following numbers are forbidden: 3, 4, 5, 9

Which means the remaining possible candidates are: 1, 2, 6, 7, 8. These become the plausible values for cell A1.

Similarly, the candidates for each cell:

	1	2	3	4	5	6	7	8	9
A	1 2 7 8	2 6 8	4	6 8	5	6 8 9	3 7 8	1 7	1 2 7 8
B	9	2 5 8	1 2 8	7	3	4	6	1 5	1 2 5 8
C	5 6 7 8	5 6 8	3	6 8	2	1	5 7 8	4	9
D	1 2	3	5	2 6	9	6 7	4	8	1 7 6
E	1 2 4 8	9	1 2 8	2 4 5 6 8	4	5 6 7 8	5 7	3	1 5 6 7
F	4 8	7	6	4 5 8 3	1	5 8 3	9	2	5
G	3	1	8	9	7	5	2	5 6	4 5 6 8
H	4 5 6 7	4 5 6	9	1	8	2	5 7	5 6 7	3
I	2 4 5 7 8	2 4 5 8	2 7 8	3 4 5	6	3 5	1	5 7 9	4 5 7 8

Step 2 — Recursive Exploration Begins

Now comes the actual Backtracking process. The algorithm tries each number one by one: $1 \rightarrow 2 \rightarrow 6 \rightarrow 7 \rightarrow 8$. Suppose we start by selecting: $A1 = 1$

	1	2	3	4	5	6	7	8	9
A	1 2 7 8	2 6	4	6 8	5	6 8 9	3 7 8	1 7	1 2 7 8
B	9	2 5 8	1 2 8	7	3	4	6	1 5	1 2 5 8
C	5 6 7 8	5 6 8	3	6 8	2	1	5 7 8	4	9
D	1 2	3	5	2 6	9	6 7	4	8	1 7 6
E	1 2 4 8	9	1 2 8	2 4 5 6 8	4	5 6 7 8	5 7	3	1 5 6 7
F	4 8	7	6	3 4 5 8	1	3 5 8	9	2	5
G	3	1	8	9	7	5	2	5 6	4 5 6 8
H	4 5 6 7	4 5 6	9	1	8	2	5 7	5 6 7	3
I	2 4 5 7 8	2 4 5 8	2 7 8	3 4 5	6	3 5	1	5 7 9	4 5 7 8

The cell (0,0) is filled with 1 → Not the final solution

Very important observation: Just because a number is *currently valid* does not mean it is part of the final solution. This is the entire philosophy of Backtracking. The solver temporarily accepts the choice and moves forward recursively to solve the next empty cell.

So the process repeats again: **Find another empty cell** → **Determine feasible values** → **Place one value** → **Continue recursively**

This continues over and over throughout the board.

Step 3 — Filling the First Sub-grid

Below is the sequence demonstrated in the illustration:

- A1 is filled with 1 → We already did this
- C1 is filled with 5

	1	2	3	4	5	6	7	8	9
A	1		4		5				
B	9			7	3	4	6		
C	5		3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

- $C2$ is filled with 6

	1	2	3	4	5	6	7	8	9
A	1		4		5				
B	9			7	3	4	6		
C	5	6	3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

- $B2$ is filled with 2

	1	2	3	4	5	6	7	8	9
A	1		4		5				
B	9	2		7	3	4	6		
C	5	6	3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

- $B3$ is filled with 8

	1	2	3	4	5	6	7	8	9
A	1		4		5				
B	9	2	8	7	3	4	6		
C	5	6	3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

At this point, everything still appears correct. But then the algorithm reaches *A2*. And suddenly, a contradiction appears, because the only possible value left for that cell is 7.

	1	2	3	4	5	6	7	8	9
A	1	7	4		5				
B	9	2	8	7	3	4	6		
C	5	6	3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

However, placing that number violates the Sudoku constraints in the column. And this is where Backtracking starts revealing its true power.

Step 4 — The Backtracking Phase

Since the current configuration leads to a conflict, the solver realizes: ***“One of the previous decisions must be incorrect”***.

So instead of restarting the puzzle entirely, the algorithm goes backward. The most recent filled cell was: $B3 = 8$. The solver removes that value and tries the next possible number.

This process is called **Backtracking**. If another number exists for $B3$, the algorithm tests it. If no numbers work, it backtracks even further.

- So eventually: $B3$ has no remaining valid numbers. Therefore, the solver returns to $B2$. Instead of placing 2, it now tries $B2 = 8$

	1	2	3	4	5	6	7	8	9
A	1		4		5				
B	9	8		7	3	4	6		
C	5	6	3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

Then the recursive process resumes again. The procedure always follows the same pattern: **Try** → **Validate** → **Continue** → **Fail** → **Undo** → **Retry**

Step 5 — Cascading Backtracking

Sometimes, changing a single cell is not enough. For example, trying $B3 = 2$ and then trying $A2 = 7$

	1	2	3	4	5	6	7	8	9
A	1	7	4		5				
B	9	8	2	7	3	4	6		
C	5	6	3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

This still creates contradictions later in the puzzle. So the algorithm is forced to backtrack even further: $C2$

Now the solver changes: $C2 = 8$

	1	2	3	4	5	6	7	8	9
A	1		4		5				
B	9			7	3	4	6		
C	5	8	3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

Then tries new possibilities again for future cells.

	1	2	3	4	5	6	7	8	9
A	1		4		5				
B	9	2	6	7	3	4	6		
C	5	8	3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

	1	2	3	4	5	6	7	8	9
A	1		4		5				
B	9	2	7	7	3	4	6		
C	5	8	3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

But even those may eventually fail. So the solver keeps retreating backward through previous decisions until a valid path is found. Here, we backtrack all

the way to $C1$, the second cell that we filled earlier in the board. And we try a different number again.

	1	2	3	4	5	6	7	8	9
A	1		4		5				
B	9			7	3	4	6		
C	5		3		2	1		4	9
D		3	5		9		4	8	
E		9						3	
F		7	6		1		9	2	
G	3	1		9	7		2		
H			9	1	8	2			3
I					6		1		

Eventually, it found a solution (but of course this solution is not found randomly, but a process of ***trying, continuing, violating the constraints, and retreating***).

	1	2	3	4	5	6	7	8	9
A	2	6	4	8	5	9	3	1	7
B	9	8	1	7	3	4	6	5	2
C	7	5	3	6	2	1	8	4	9
D	1	3	5	2	9	7	4	8	6
E	8	9	2	5	4	6	7	3	1
F	4	7	6	3	1	8	9	2	5
G	3	1	8	9	7	5	2	6	4
H	6	4	9	1	8	2	5	7	3
I	5	2	7	4	6	3	1	9	8

This is why Backtracking is often visualized using a **State Space Tree**:

- Every node represents a decision
- Every branch represents a possible number
- Invalid branches are abandoned
- Valid branches continue expanding

For simplicity, however, you can think of it as repeatedly testing plausible values for each cell until the board stabilizes into a complete solution.

Summary

Sudoku is a game, but a Sudoku Solver is an algorithmic problem.

Instead of solving the puzzle using human intuition alone, computers require a systematic strategy capable of exploring possibilities while respecting the constraints of the board. This is where Backtracking becomes extremely effective.

The core idea of Backtracking is simple: find an empty cell, determine all feasible numbers, attempt one possibility, continue recursively, and backtrack whenever a contradiction appears.

Rather than restarting the entire puzzle after making a mistake, the algorithm intelligently returns to previous decisions, removes invalid choices, and explores alternative paths. This recursive trial-and-error process continues until the puzzle is completely solved.

Throughout the procedure, the solver constantly checks three important Sudoku conditions: each number must appear only once in every row, only once in every column, and only once in every 3×3 sub-grid.

Although Backtracking may initially seem inefficient, it becomes surprisingly powerful when combined with good strategies, such as prioritizing cells with fewer possible candidates. In practice, this greatly reduces unnecessary recursive calls and improves the solving process.

So in the end, Sudoku Solver is not about “guessing randomly.” It is about systematically exploring a decision space, eliminating impossible states, and gradually converging toward a valid solution.

And this is what makes Backtracking such an elegant algorithm: ***It transforms a seemingly impossible puzzle into a structured search problem that computers can solve step by step.***

References

<https://www.sudoku-solutions.com/>

<https://www.geeksforgeeks.org/dsa/sudoku-backtracking-7/>

<https://www.kaggle.com/code/mexwell/sudoku-solver-a-backtracking-approach>

<https://www.youtube.com/watch?v=rctRBF7wmg>